Intelligent Publisher Website Classifier

Overview

At Parity Cube, we operate affiliate marketing platforms that work with thousands of publisher websites. We need to automatically evaluate publisher websites to determine their suitability for campaigns — flagging low-quality, irrelevant, or policy-violating sites before they go live.

Your task is to build a working API service that classifies publisher websites across multiple health and quality dimensions. This mirrors real problems we solve internally.

Time: 2–3 days
Language: Python
Stack: FastAPI, LLMs, RAG / Vector DB

Core Task

Build a FastAPI service with the following endpoint:

`POST /classify`

Request body:

```
{ "url": "https://example.com" }
```

Response:

```
{
  "url": "https://example.com",
  "is_cashback_site": false,
  "is_adult_content": false,
  "is_gambling": false,
  "is_agency_or_introductory": false,
  "is_scam_or_low_quality": false,
  "overall_score": 85,
  "summary": "Brief explanation of classification",
  "confidence": "high"
}
```

Requirements

Part 1 — Core Classification *(mandatory)*

- Fetch and parse the website content (handle JS-heavy sites gracefully)
- Use an LLM of your choice (GPT-4o-mini, Claude Haiku, Groq Llama, etc.) to classify across all dimensions
- Return structured output with a brief explanation
- Handle errors gracefully (unreachable sites, timeouts, bot-blocking)

Part 2 — RAG Layer *(mandatory)*

- Design and implement a RAG-based **"policy store"** — a small vector database (ChromaDB or FAISS) containing classification rules and guidelines
- You define the rules; make them sensible and production-realistic
- At classification time, retrieve relevant policy chunks and inject them into the LLM prompt
- **In your README, explain:**
  - Your chunking strategy and why you chose it
  - How you decided what to put in the policy store vs. directly in the prompt

Part 3 — Agentic Enhancement *(bonus)*

Redesign Part 1 as a **2-agent pipeline:**

- **Agent 1 — Researcher:** Fetches the site, extracts key signals (content type, traffic indicators, language, categories, etc.)
- **Agent 2 — Classifier:** Uses retrieved policies + extracted signals to make the final decision

Use CrewAI, LangGraph, or plain async orchestration — your choice.

In your README, reflect on: *is the agentic approach meaningfully better here? Why or why not?*

Part 4 — Production Thinking *(written, not coded)*

Answer these briefly in your README or a separate doc:

1. This API will process **500 publisher URLs per day**. What breaks first? How do you fix it?
2. GPT-4o-mini gives **78% accuracy** on your test set. What are your next 3 experiments to improve it?
3. A publisher complains their site was **incorrectly flagged as adult content**. How do you handle this operationally?

Deliverables

- Working code in a GitHub repo (or zipped folder)
- `README.md` with setup instructions and design decisions
- Sample output for **at least 5 URLs** (mix of good and policy-violating sites)
- Brief write-up for Part 4

Evaluation Criteria

Area	What We're Looking For
RAG Design	Chunking strategy, retrieval quality, prompt injection
LLM Prompting	Structured output, edge case handling, prompt quality
Code Quality	Clean FastAPI, error handling, async where appropriate
Production Awareness	Not just "it works" but "it scales and fails gracefully"
Part 4 Depth	This is where senior vs. junior judgment shows up

---

Notes

- Don't over-engineer. A clean, working solution beats a half-finished complex one.

- You can use any open-source libraries.
- If you can't scrape a specific site (bot protection, etc.), handle it gracefully and document why.
- We're not evaluating for 100% classification accuracy — we're evaluating your **approach and thinking**.
- If you have questions about the assignment, reach out to the HR team and they'll connect you with the right person.

Please submit your solution within the agreed timeframe. Good luck!

\vspace{1cm} *Parity Cube Pvt Ltd | hr@paritycube.com*